```python
"""

Handles all communication with external APIs.
  Open-Meteo Geocoding API   convert city name → lat lon
   Open-Meteo Weather API     fetch current + 7 days forecast

Both APIs are completely free and does not API key.

"""

import urllib.request
import urllib.parse
import json
from datetime import date, timedelta
from typing import Optional

from models import WeatherRecord, FavoriteCity
from exceptions import APIError, CityNotFoundError


# Helper - thin HTTP GET wrapper

def _get_json(url: str) -> dict:
    """
    Fetch JSON from *url* using only the standard library.

    Raises:
        APIError: on HTTP or parsing failures.
    """
    try:
        with urllib.request.urlopen(url, timeout=10) as response:
            raw = response.read().decode("utf-8")
            return json.loads(raw)
    except urllib.error.HTTPError as exc:
        raise APIError(f"HTTP error when calling {url}", status_code=exc.code) from exc
    except urllib.error.URLError as exc:
```

```python
        raise APIError(f"Network error: {exc.reason}") from exc
    except json.JSONDecodeError as exc:
        raise APIError("Failed to parse API response as JSON") from exc


# GeocodingClient


class GeocodingClient:
    """Converts a city name to geographic coordinates via Open-Meteo."""

    BASE_URL = "https://geocoding-api.open-meteo.com/v1/search"

    def search(self, city_name: str) -> FavoriteCity:
        """
        Return a FavoriteCity with lat lon for *city_name*.

        Raises:
            CityNotFoundError: if no results are returned.
            APIError:          on network / HTTP problems.
        """
        params = urllib.parse.urlencode({
            "name": city_name,
            "count": 1,
            "language": "en",
            "format": "json",
        })
        url = f"{self.BASE_URL}?{params}"
        data = _get_json(url)

        results = data.get("results")
        if not results:
            raise CityNotFoundError(city_name)

```

```python
        first = results[0]
        return FavoriteCity(
            name=first.get("name", city_name),
            country=first.get("country", ""),
            latitude=float(first["latitude"]),
            longitude=float(first["longitude"]),
        )


# WeatherClient

# Mapping WMO weather-code → human description
_WMO_CODES: dict[int, str] = {
    0: "Clear sky", 1: "Mainly clear", 2: "Partly cloudy", 3: "Overcast",
    45: "Fog", 48: "Depositing rime fog",
    51: "Light drizzle", 53: "Moderate drizzle", 55: "Dense drizzle",
    61: "Slight rain", 63: "Moderate rain", 65: "Heavy rain",
    71: "Slight snow", 73: "Moderate snow", 75: "Heavy snow",
    80: "Slight showers", 81: "Moderate showers", 82: "Violent showers",
    95: "Thunderstorm", 96: "Thunderstorm w/ hail", 99: "Thunderstorm w/ heavy hail",
}


class WeatherClient:
    """Fetches current weather and 7-day forecast from Open-Meteo."""

    BASE_URL = "https://api.open-meteo.com/v1/forecast"

    def _build_url(self, lat: float, lon: float) -> str:
        params = urllib.parse.urlencode({
            "latitude": lat,
            "longitude": lon,
```

```
            "current": ",".join([
                "temperature_2m",
                "apparent_temperature",
                "relative_humidity_2m",
                "wind_speed_10m",
                "weather_code",
            ]),
            "daily": ",".join([
                "temperature_2m_max",
                "temperature_2m_min",
                "weather_code",
                "wind_speed_10m_max",
                "precipitation_sum",
            ]),
            "wind_speed_unit": "kmh",
            "timezone": "auto",
            "forecast_days": 7,
        })
        return f"{self.BASE_URL}?{params}"

    def fetch(self, city: FavoriteCity) -> tuple[WeatherRecord, list[WeatherRecord]]:
        """
        Return (current_record, forecast_list) for *city*.

        forecast_list contains one WeatherRecord per day for 7 days.

        Raises:
            APIError: on network / HTTP problems.
        """
        lat, lon = city.coordinates
        url = self._build_url(lat, lon)
        data = _get_json(url)


        cur = data["current"]
        code = int(cur.get("weather_code", 0))
        current_record = WeatherRecord(
            city=city.name,
```

```
            country=city.country,
            recorded_date=date.today(),
            temperature_c=round(float(cur["temperature_2m"]), 1),
            feels_like_c=round(float(cur["apparent_temperature"]), 1),
            humidity_pct=int(cur["relative_humidity_2m"]),
            wind_speed_kmh=round(float(cur["wind_speed_10m"]), 1),
            weather_description=_WMO_CODES.get(code, "Unknown"),
            is_forecast=False,
        )


        daily = data["daily"]
        forecast_list: list[WeatherRecord] = []

        for i, day_str in enumerate(daily["time"]):
            day_date = date.fromisoformat(day_str)
            day_code = int(daily["weather_code"][i])
            avg_temp = round(
                (float(daily["temperature_2m_max"][i]) + float(daily["temperature_2m_min"][i])) / 2,
                1,
            )
            record = WeatherRecord(
                city=city.name,
                country=city.country,
                recorded_date=day_date,
                temperature_c=avg_temp,
                feels_like_c=avg_temp,
                humidity_pct=0,
                wind_speed_kmh=round(float(daily["wind_speed_10m_max"][i]), 1),
                weather_description=_WMO_CODES.get(day_code, "Unknown"),
                is_forecast=True,
            )
            forecast_list.append(record)
```

```python

        return current_record, forecast_list
```